Pothi: a bibliography that belongs to the paper

A local, manuscript-first reference manager for people who write.

Kanchan Sarkar

Correspondence: pcksarkar@gmail.com

Article type: Tool description | Date: May 10, 2026

Abstract Reference management software is mature. Zotero, Mendeley, EndNote, and JabRef each solve real problems, and many laboratories run on them for years without trouble. Yet researchers still write papers as folders, not as queries against a central library. A working manuscript folder typically holds a draft, figures, data exports, and a bibliography. The first three live with the paper. The last one usually lives somewhere else: in a desktop application, on a sync server, or behind a vendor login. **Pothi** treats the manuscript as the unit of organisation. Each manuscript carries its own ordered bibliography, written next to the draft as a plain `references.bib` (and a CSL-JSON copy), and the global library is the union of those manuscript bibliographies. Metadata is fetched from CrossRef, OpenAlex, and other public sources when a DOI, arXiv ID, ISBN, or URL is supplied; Pandoc is the bridge for Word. Pothi has no account system, server, or subscription layer; its durable library is plain JSON in a folder chosen by the user. This note describes the problem Pothi addresses, the design decisions behind it, the normal workflow, how it compares with established reference managers, and its present limits. Pothi is not meant to replace any established reference manager. It is meant for the narrower case of keeping the bibliography close to the manuscript, in files the author can inspect, edit, share, version-control, and preserve.

Why I built this

A research career produces a quiet archive: project folders, drafts, figures, data exports, and the references that anchor each paper to the literature. Over time, the references are the part most likely to drift. PDFs end up scattered across project folders, cloud drives, old laptop backups, and the occasional shared drive nobody can find anymore. This is not laziness. It is what happens when a long sequence of small, reasonable decisions accumulates without an explicit policy for where the bibliography of each paper lives. Most groups I have worked with have some version of this problem, and most have made peace with it.

I write papers in LaTeX, and sometimes in Word. Over the years I have used Zotero with Better BibTeX, JabRef, Mendeley, and EndNote. Each is a serious tool, written by people who have thought carefully about citation. None of them quite reproduces the workflow I want, which is straightforward to state: I want the references for a manuscript to live in the manuscript's folder, in a file I can read in any text editor, version-control with the rest of the paper, and send to a co-author who happens to use a different reference manager.

For a working manuscript, the global library is usually not the main object. The important object is the smaller set of references that belong to that paper: why each reference is cited, where it enters the argument, and whether the final bibliography can be sent to a co-author without requiring the same reference-manager setup.

Zotero, Mendeley, EndNote, and JabRef each solve real problems. I have used them, and I do not dismiss them. They are mature tools with users who have good reasons for staying with them. Still, they did not quite fit the workflow I wanted.

- **The paper should be the unit of organization.** A global library is useful for search and reuse, but a manuscript is written from a smaller, deliberate reference set. That set should live with the manuscript.
- **The bibliography should be a plain file.** For a LaTeX paper, `references.bib` belongs in the folder from which the paper is compiled. It should be visible, editable, easy to version-control, and easy to send.
- **The local copy should remain useful without the program.** If a reference manager is abandoned, changes policy, or syncs badly, the paper's bibliography should not be trapped inside it.
- **Attachments should be robust.** Moving a project folder or renaming a PDF should not break the record of what the paper cites.
- **The tool should stay understandable.** A researcher should be able to inspect the files and understand the workflow without needing to become an unpaid systems administrator. Humanity has already invented enough of those roles by accident.

Pothi came from that practical irritation. It is not a commercial platform and it is not trying to become one. It is a small browser-based tool written for my own writing practice and released because the same practice may help other researchers. If Zotero with Better BibTeX already fits a lab's work, there is no reason to move. The point is not tool loyalty. The point is whether the bibliography remains close to the paper and recoverable outside the application.

Design principles

Five decisions shaped the program and kept its scope from drifting into another large reference system with a smaller logo.

Local first

The data stays on the user's machine. Pothi has no login, no cloud service, and no subscription layer. The full library is written as plain JSON into a folder selected by the user. The browser's IndexedDB is used as a working cache, not as the only copy of the record. This follows the local-first argument made by Kleppmann and colleagues [1]: software should let users retain ownership of their data, continue working without depending on a central service, and remain usable when the vendor or the network is gone. The same logic underwrites the FAIR principles for research data [2]: a record that the user does not control cannot be reliably findable, accessible, interoperable, or reusable on the long timescales of scholarship.

Manuscript first

A reference matters because it supports a piece of writing. Pothi therefore treats the manuscript as a first-class object. Each manuscript has a name, a folder, an ordered bibliography, and a short note on why each reference is cited. The global library is the union of those manuscript bibliographies, plus any references the user has collected but not yet assigned.

Plain text on disk

When a folder is linked, Pothi writes `_pothi-library.json` for the whole library and writes `references.bib` and `refs.json` for each linked manuscript. These files can be opened in a text editor, committed to git, copied to a collaborator, or backed up with ordinary file-sync tools. Plain text is not a lifestyle choice here; it is a recovery strategy. The case for plain-text scientific data and tooling has been made repeatedly [3, 4], and the same logic applies to a paper's bibliography: a format that the user can inspect and repair by hand outlasts the application that produced it.

Schema as configuration

Entry types, required fields, and citekey patterns live in `js/schema.js`. Adding a field or changing a citekey template means editing one configuration file. Pothi does not need a plugin system for simple local customization.

No build step

The application is HTML, CSS, and ES module JavaScript. The vendored libraries are committed in `vendor/`. There is no `npm install`, no bundler, and no transpilation step. Edit a file, refresh the browser, and the change is visible. This is not fashionable software engineering, but it is serviceable. Fashion can remain busy elsewhere, as it usually does.

How it is put together

The project is a folder of static files.

```
pothi/ # repository root
  README.md # user manual
  install.sh # Linux / macOS launcher installer
  install.bat # Windows launcher installer
  app/ # the application (served over local HTTP)
    index.html # shell + import map
    css/
      variables.css # design tokens
      base.css, layout.css, components.css
    js/
      schema.js # entry types + fields + citekey template
      db.js # IndexedDB wrapper (entries, manuscripts, meta)
      bibtex.js # parser + emitter, with LaTeX -> Unicode
      citekey.js # {Author1}{year}{title3} template engine
      lookup.js # CrossRef + OpenAlex + arXiv + OpenLibrary + S2
      dedup.js # title+author fuzzy duplicate detection
      folder.js # File System Access API wrapper
      folder-scan.js # recursive walk + SHA-256 dedup
      pdf-extract.js # PDF.js + DOI/arXiv regex
      pdf-preview.js # in-app PDF viewer
      docx.js # Pandoc-bridge equivalent (fflate + DOMParser)
      styles.js # author-year + numeric citation styles
      library-backup.js # export/import + auto-write to folder
      smart-collections.js # saved searches
      manuscripts.js # manuscript helpers
      markdown.js # tiny safe Markdown -> DOM
      tag-suggest.js # TF-IDF tag suggestions
      app.js # main render
    vendor/ # Preact, htm, PDF.js, fflate (vendored)
    LICENSE # MIT
```

The browser starts from `index.html`. An import map points each module name to a local vendored file:

```
<script type="importmap">
{ "imports": {
    "preact": "./vendor/preact.module.js",
    "preact/hooks": "./vendor/preact-hooks.module.js",
    "htm": "./vendor/htm.module.js"
  } }
</script>
```

IndexedDB holds three stores: `entries`, `manuscripts`, and a `meta` key-value store. The `meta` store keeps the linked-folder `FileSystemDirectoryHandle`, allowing the browser to remember the folder after reloads when permissions allow it.

Modern browsers do not run ES modules directly from `file://`. Pothi therefore uses a small local HTTP server. The supplied installers add a `pothi` command to the user's `PATH`. Running `pothi` starts Python's `http.server` on `127.0.0.1:8765` and opens the browser at that address.

A normal working day with Pothi

This section describes the tasks that matter in ordinary use. It is not a button-by-button manual. Nobody became a better researcher by reading another UI inventory, despite the best efforts of software documentation.

Add a paper by DOI, arXiv ID, ISBN, or URL

Click + *Add reference*. Paste a DOI, a `doi:` prefix, a DOI URL, an arXiv ID, an ISBN, or a journal URL that contains a DOI. Pothi identifies the input and sends it to the relevant public source: CrossRef for most DOI records, arXiv through DOI metadata where available, and OpenLibrary for books. Once the primary record returns, Pothi enriches it with abstract and citation count from OpenAlex, and falls back to Semantic Scholar where OpenAlex does not carry the metric. Each enrichment step soft-fails so a single rate-limited service does not block the import.

The usual fields are filled automatically: title, authors in BibTeX name form, journal, volume, issue, pages, year, and DOI. A citekey is generated from the configured template, by default `{Author1}{year}{title3}`.

Duplicate detection runs at insert time. Strict equality on DOI or ISBN is the obvious case. The harder case is the same paper added once from a preprint and once from the journal version, or once from a `.bib` import without a DOI. Pothi normalises the title (lowercased, punctuation stripped) and combines it with the publication year and a token-set of author surnames. When a fuzzy match is found, the user is offered two outcomes: merge the new metadata into the existing entry (filling only empty fields, never overwriting user edits) or add as a fresh duplicate anyway.

Search the web by keyword or author name

The toolbar's *Search web. . .* button opens a modal that queries CrossRef and OpenAlex in parallel, dedupes by DOI, and ranks results by OpenAlex citation count. Results carry source badges so the user can see which service contributed each hit. Each row has an + *Add to library* button that pipes the result through the same fuzzy-dedup path used for manual additions.

Google Scholar has no public API and Web of Science is paywalled, neither of which is reachable from a browser without a key. CrossRef and OpenAlex together cover roughly four hundred million records and most of what Scholar would surface for a given query.

The mode picker beside the search input handles a relevance pitfall: a query like *kanchan sarkar* run as a generic search re-

turns every paper that mentions either token in title or abstract. Pothi auto-detects name-shaped queries (two to four alphabetic tokens, no research-prose stopwords) and routes them to author-specific endpoints (`query.author=` on CrossRef, `filter=raw_author_name.search:` on OpenAlex). The user can override to *Anywhere*, *Author*, or *Title* mode explicitly.

Refresh missing metadata across the library

Researchers who import an old `.bib` file usually inherit a long tail of entries with no abstract and no citation count. The *Refresh missing metadata* action under the Export menu walks every entry whose DOI (or DOI-bearing URL) is present and whose abstract or citation count is empty. It calls CrossRef and OpenAlex serially with a small throttle, persists each filled entry as it goes so partial progress survives, and shows a progress banner with a cancel button. A library of several hundred entries imported flat from a `.bib` file becomes a library with abstracts and citation counts in a few minutes.

Drop in a PDF

A PDF can be dragged anywhere onto the page. Pothi then:

1. extracts embedded text and metadata using PDF.js;
2. searches the early pages for a DOI or arXiv identifier;
3. retrieves metadata when a match is found;
4. generates a citekey;
5. copies the PDF into the linked library folder as `<citekey>.pdf` when folder access is enabled.

For journal articles this works often because the DOI usually appears in the first pages. For PDFs without a DOI, Pothi uses what it can extract from file metadata and leaves the user to complete the record. If a new PDF has the same content hash as a file already attached to an entry, Pothi does not import it again.

Build a bibliography for a manuscript

Create a manuscript from the sidebar and give it a name, for example *Polymorphism review 2026*. The manuscript view has a search box and a bibliography list. Search the library, add the references that belong to the paper, and arrange them in whatever order is useful while drafting.

Each cited reference has space for a rationale. The instruction is intentionally simple: record why the reference is present. A note such as "cited in §3.2 because Bernstein defines the standard polymorphism framework" is enough. Months later, that sentence can spare an author or co-author from reconstructing a citation decision from memory, which is where good intentions go to die quietly.

A manuscript can be linked to the folder that contains its draft. Once linked, Pothi writes `references.bib` and `refs.json` into that folder after each change, with a short debounce so the disk is not rewritten on every keystroke.

For LaTeX.. A linked manuscript folder can be used directly:

```
% In your manuscript folder, after linking it to the manuscript:
\documentclass{article}
\usepackage{biblatex}
\addbibresource{references.bib} % Pothi keeps this current

\begin{document}
As Bernstein argues~\cite{Bernstein2020Polymorphism}, ...
% Or for vanilla BibTeX:
% \bibliography{references}
\printbibliography
\balance
\end{document}
```

Run `pdflatex`, `latexmk`, or the usual local build command. When the cited list changes in Pothi, the `references.bib` file beside the manuscript updates within a few seconds. The next compile sees the updated bibliography. There is no export step and no word-processor plugin mediating the relationship. The plain `.bib` file is the contract; that contract has been the canonical bibliography exchange format for nearly four decades [5] and remains the lowest common denominator across BibTeX, biblatex, and most Pandoc workflows.

For BibLaTeX users, the emitter is designed to preserve Unicode and brace protection. For traditional BibTeX users, the normal `inputenc` setup continues to apply.

Write a Word document with citation markers

For Word, Pothi uses simple citation markers in the document text:

- `[@SmithCrystal2024]` for one citation;
- `[@Smith2024; @Doe2023]` for several citations;
- `[@Smith2024, p. 42]` for a citation with a page locator.

Drag the `.docx` file onto Pothi. The program unzips the document, walks through `word/document.xml`, replaces each marker with a formatted citation in the selected style, and appends a bibliography. The output is saved as `<name>-cited.docx`. Missing citekeys are shown as `[?]` in the document and listed after processing.

This is not a live Word add-in. It is a drop-and-replace workflow. That limit is deliberate in the present version. A full Office add-in would require a separate deployment model, an Office.js task pane, and additional state management. Those may be worth doing later, but they are not required for the core manuscript-first workflow. For users who already have Pandoc [6] on their system, the same exported CSL-JSON drives a much wider range of citation styles via `pandoc -citeproc`.

Recover after changing browsers, ports, or machines

Browser storage is tied to an origin. To the browser, `http://127.0.0.1:8765` and `http://127.0.0.1:8766` are different places, even though a normal person would call both "the thing running on my laptop." The browser is technically correct, which is the most tiresome kind of correct.

Pothi's answer is to keep the durable copy on disk. When a library folder is linked, Pothi writes `_pothi-library.json` after each change. On a fresh browser origin or a new machine, the user links the same folder. Pothi detects the backup, reports how many entries it found and when it was last saved, and asks whether to restore it into IndexedDB. The disk file remains the source of truth; the browser cache can be rebuilt.

Reverse the wrong action

Single-row delete and bulk delete both leave a ten-second window with an *Undo* button on the toast. Inside that window the deleted record is restored to disk and to the visible list. Beyond it the deletion is permanent. The pattern matters because most researchers will eventually delete the wrong row in a long bibliography, and a confirmation dialog alone is not enough to rebuild trust after the first accident.

	Zotero+BBT	JabRef	Mendeley	EndNote	Pothi
Free / open-source	yes / yes	yes / yes	yes / no	no / no	yes / yes
No account required	yes (BBT)	yes	no	no	yes
Local-only by default	no	yes	no	yes	yes
Plain text on disk	no	yes (.bib)	no	no	yes (JSON + .bib)
Manuscript-first model	no	no	no	no	yes
Per-citation rationale	no	no	no	no	yes
`.bib` lives next to manuscript	opt-in (BBT)	yes	no	no	yes
Live-sync `.bib` on edit	on save (BBT)	on save	no	no	yes (4 s)
Watch-folder auto-import	via ZotFile	no	no	no	yes
Drag-PDF DOI sniff	yes	no	yes	no	yes
Fuzzy duplicate detection	yes	on import	yes	yes	yes (title+author+year)
Keyword search across CrossRef + OpenAlex	no	no	no	no	yes
Author-mode auto-detect	no	no	no	no	yes
Library-wide metadata refresh	manual	manual	manual	manual	one click
Undo for delete	yes	no	yes	yes	yes (10 s toast)
docx “cite-while-write”	yes (Word add-in)	no	yes	yes	via Pandoc bridge
Custom entry types	via plugin	yes	no	yes	via schema.js
Runs in browser	web only	no	no	no	yes
Install ceremony	app + ext + plugin	app + Java	app + ext	app	none (browser)
Lines of source	~300 k	~200 k	closed	closed	~10 k

Table 1. A practical comparison with common reference managers. The entries reflect the author’s current reading of the tools and should be corrected if wrong.

Move through the list without the mouse

Arrow keys walk the visible list, opening the detail pane as the cursor moves. `Enter` selects the first row when nothing is selected. The slash key focuses the search input from anywhere outside a text field. `Escape` clears the search, then closes the detail, then clears the bulk selection in that order. The intent is the same as the rest of the program: keep the user near their work, not navigating it.

An honest comparison

The table should be read as a scope statement, not as a victory lap. Zotero with Better BibTeX is probably the better choice for many researchers, especially groups that need mature sync, browser capture, and established community support. EndNote remains embedded in many institutions. JabRef is a strong fit for BibTeX-centered workflows.

Pothi is narrower. It is for the researcher who wants a manuscript folder to contain the draft, figures, bibliography, and citation notes in readable local files. That is not everyone’s need, but it is a real one.

What it does not do

The limits are worth stating plainly.

- **No live Microsoft Word add-in.** Word support is based on a drop-and-replace workflow for `.docx` files. A real Office add-in would be a separate project.
- **Folder linking requires a Chromium-based browser.** The File System Access API is available in browsers such as Chrome, Edge, Brave, and Opera. It is not available in Firefox or Safari at the time of writing. Basic reference entry, browsing, search, and export can work in modern browsers without folder linking.
- **No built-in cloud sync.** Use Nextcloud, Dropbox, iCloud, git, or another sync system on the linked folder. The synchronization unit is the plain JSON backup file. File conflicts remain the responsibility of the sync system and the user.
- **No real-time multi-user editing.** Two tabs or two machines can produce stale views or file conflicts. Pothi is local-first, not collaborative in the Google Docs sense.
- **Single maintainer.** I wrote the tool. The source is small and MIT-licensed, so it can be forked, but it does not have an institutional team behind it.
- **Limited citation styles in the built-in docx path.** The current pipeline includes author-year and numeric styles. For full CSL support today, Pandoc remains the better path.
- **Best-effort citation metadata.** Semantic Scholar and other public services may rate-limit requests or return incomplete records. Pothi should make correction easy, not pretend that public metadata is clean by default.

Repository, installation, and license

Code.. `https://github.com/kanchansarkar/pothi`

License.. MIT. The repository’s `LICENSE` file contains the full text. Use, modify, and redistribute the code under those terms.

Install.. Clone the repository. From the project root:

```
./install.sh # Linux / macOS
install.bat # Windows
```

The repository’s top level holds the `README.md` and the installers; the application itself lives in the `app/` subfolder. The installer adds a `pothi` command to the user’s `PATH` and creates a launcher: `pothi.desktop` on Linux, `Pothi.command` on macOS, or `Pothi.bat` on Windows. After restarting the shell, run `pothi`. The local server starts and the browser opens.

Manual run.. To run without installing:

```
cd app && python3 -m http.server 8765 --bind 127.0.0.1
```

Then open `http://127.0.0.1:8765/` in a Chromium-based browser.

Future work and invitations

The useful next steps are clear enough:

- a real Microsoft Word add-in using Office.js;
- PDF annotation extraction, so highlights and notes can be attached to references;
- bring-your-own CSL support through a local citeproc implementation;
- integration with field-specific research databases, such as crystallographic or experimental records;
- a simpler first-run tour and a bundled double-click launcher for users who should not have to care what a local HTTP server is.

The most useful contributions will be concrete: bug reports with reproduction steps, examples of imported `.bib` files that fail, PDFs whose metadata is misread, and screenshots of browser-specific layout problems. Pull requests are welcome, especially when they keep the program small and the data readable.

Acknowledgements

Pothi depends on Preact, htm, PDF.js, fflate, and the public APIs of CrossRef, OpenAlex, Semantic Scholar, and OpenLibrary. The local-first framing draws directly on Kleppmann *et al.* [1]; the plain-text emphasis on Broman and Woo [3] and Wilson *et al.* [4]; the data-stewardship framing on the FAIR principles of Wilkinson *et al.* [2]; the bibliography exchange format on Patashnik [5]; and the Word workflow on Pandoc [6]. The manuscript-first emphasis comes from my own writing practice: a paper is not only an output file but a folder of decisions that should remain legible after the software changes.

References

[1] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. F. McGranaghan, "Local-first software: you own your data, in spite of the cloud," in *Proc. 2019 ACM SIGPLAN Int. Symp. on New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*, ACM, 2019, pp. 154–178. Essay form: `https://www.inkandswitch.com/local-first/`.

[2] M. D. Wilkinson *et al.*, "The FAIR Guiding Principles for scientific data management and stewardship," *Scientific Data*, vol. 3, art. 160018, 2016.

[3] K. W. Broman and K. H. Woo, "Data organization in spreadsheets," *The American Statistician*, vol. 72, no. 1, pp. 2–10, 2018.

[4] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal, "Good enough practices in scientific computing," *PLoS Computational Biology*, vol. 13, no. 6, art. e1005510, 2017.

[5] O. Patashnik, *BibTeXing*, technical document distributed with the BibTeX program, 1988. Available: `https://ctan.org/pkg/bibtex`.

[6] J. MacFarlane, *Pandoc: a universal document converter*, software, `https://pandoc.org/` (accessed May 2026).